

PyDiag

Challenge Pratique Python

Listes, Dictionnaires & Sets

Notions évaluées

Listes, dictionnaires, sets, compréhensions,
combinaison des structures

Table des matières

Contexte	2
Mission de l'apprenant	2
1 Bloc 1 — Listes (35 min — 20 pts)	2
2 Bloc 2 — Dictionnaires (40 min — 30 pts)	3
3 Bloc 3 — Sets (25 min — 20 pts)	5
4 Bloc 4 — Combinaison listes / dicts / sets (40 min — 30 pts)	6
5 Règles de Gestion	7
6 Jeu de données de test (exemple global)	8
7 Structure attendue du script	8
8 Livrables attendus	9

Contexte

Ce challenge évalue la maîtrise des trois structures de données fondamentales de Python : **listes**, **dictionnaires** et **sets**. Il est organisé en 4 blocs progressifs, chacun ciblant une structure précise, avant un dernier bloc qui les combine dans des cas proches de situations réelles (ventes, stocks, notes par matière).

L'objectif est d'évaluer la capacité de l'apprenant à :

- Choisir la structure de données adaptée à un problème donné
- Manipuler ces structures avec les méthodes et opérateurs appropriés
- Utiliser les compréhensions de liste et de dictionnaire à bon escient
- Combiner plusieurs structures pour résoudre un problème composite

Mission de l'apprenant

Pour chaque exercice, vous devez écrire une fonction ou un script Python autonome, testé avec le jeu de données fourni dans l'encadré « Exemple » correspondant. Respectez les bonnes pratiques : nommage clair (snake_case), fonctions courtes à responsabilité unique, code lisible.

1 Bloc 1 — Listes (35 min — 20 pts)

Bloc 1 : Listes

- ☐ Trouver le maximum et le minimum d'une liste sans `min()/max()`
- ☐ Filtrer une liste selon un seuil
- ☐ Compter les occurrences d'éléments sans `Counter`
- ☐ Inverser une liste sans `reverse()` ni `[::-1]`
- ☐ Fusionner deux listes triées sans `sorted()`
- ☐ Utiliser une compréhension de liste

Exemple — Données & résultat attendu

1.1 — Max/min sans fonctions natives

```
notes = [12, 18, 7, 15, 9, 20, 3, 14]
```

Résultat attendu :

```
Note max : 20
Note min : 3
```

Exemple — Données & résultat attendu

1.2 — `notes_au-dessus(notes, seuil)`

```
notes = [8, 14, 6, 17, 11, 20]
seuil = 12
```

Résultat attendu :

```
[14, 17, 20]
```

Exemple — Données & résultat attendu**1.3 — Comptage d'occurrences**

```
fruits = ["pomme", "banane", "pomme", "orange", "banane", "pomme"]
```

Sortie attendue :

```
pomme : 3  
banane : 2  
orange : 1
```

Exemple — Données & résultat attendu**1.4 — Inversion manuelle**

```
liste = [1, 2, 3, 4, 5]
```

Résultat attendu :

```
[5, 4, 3, 2, 1]
```

Exemple — Données & résultat attendu**1.5 — Fusion de deux listes triées**

```
liste_a = [1, 4, 7]  
liste_b = [2, 3, 8, 9]
```

Résultat attendu :

```
[1, 2, 3, 4, 7, 8, 9]
```

Exemple — Données & résultat attendu**1.6 — Compréhension de liste**

```
nombres = [3, 12, 7, 25, 8, 19, 2]
```

Résultat attendu (carrés des nombres pairs) :

```
[144, 64, 4]
```

2 Bloc 2 — Dictionnaires (40 min — 30 pts)

Bloc 2 : Dictionnaires

- ☐ Fonction `vendre(stock, produit, quantite)` avec vérification de disponibilité
- ☐ Fonction `produits_epuises(stock)`
- ☐ Agrégation par clé à partir d'une liste de commandes
- ☐ Inversion d'un dictionnaire (clés ↔ valeurs)
- ☐ Compréhension de dictionnaire

☐ Dictionnaires imbriqués**Exemple — Données & résultat attendu****2.1 — vendre(stock, produit, quantite)**

```
stock = {"pommes": 50, "bananes": 30, "oranges": 0}
vendre(stock, "pommes", 20)
vendre(stock, "oranges", 5)
```

Comportement attendu :

```
Vente enregistrée : 20 pommes.
Stock insuffisant pour oranges (disponible : 0).
```

État final de stock :

```
{"pommes": 30, "bananes": 30, "oranges": 0}
```

Exemple — Données & résultat attendu**2.2 — produits_epuises(stock)**

```
stock = {"pommes": 30, "bananes": 0, "oranges": 0, "kiwis": 12}
```

Résultat attendu :

```
["bananes", "oranges"]
```

Exemple — Données & résultat attendu**2.3 — Total par client**

```
commandes = [
    {"client": "Ali", "produit": "pommes", "quantite": 5},
    {"client": "Sara", "produit": "bananes", "quantite": 10},
    {"client": "Ali", "produit": "oranges", "quantite": 2},
]
```

Résultat attendu (total_par_client) :

```
{"Ali": 7, "Sara": 10}
```

Exemple — Données & résultat attendu**2.4 — Inversion d'un dictionnaire**

```
d = {"a": 1, "b": 2, "c": 3}
```

Résultat attendu :

```
{1: "a", 2: "b", 3: "c"}
```

Exemple — Données & résultat attendu**2.5 — Compréhension de dictionnaire**

```
mots = ["chat", "elephant", "abeille", "riz"]
```

Résultat attendu :

```
{"chat": 4, "elephant": 8, "abeille": 8, "riz": 3}
```

Exemple — Données & résultat attendu**2.6 — Dictionnaires imbriqués**

```
entreprise = {  
    "IT": ["Ali", "Sara", "Omar"],  
    "RH": ["Lina"],  
    "Ventes": ["Karim", "Yasmine", "Nadia", "Hicham"],  
}
```

Sortie attendue :

```
IT : 3 employe(s)  
RH : 1 employe(s)  
Ventes : 4 employe(s)
```

3 Bloc 3 — Sets (25 min — 20 pts)

Bloc 3 : Sets

- ☐ Intersection, union et différence de deux ensembles
- ☐ Détection de doublons avec un set
- ☐ Construction d'un set unique à partir de listes imbriquées
- ☐ Comprendre les limites des sets (éléments non hashables) et l'alternative avec des tuples

Exemple — Données & résultat attendu**3.1 — Intersection, union, différence**

```
atelier_python = ["Ali", "Sara", "Lina", "Karim"]  
atelier_java = ["Sara", "Omar", "Lina", "Yasmine"]
```

Résultats attendus :

```
Inscrits aux deux ateliers : {"Sara", "Lina"}  
Inscrits a au moins un atelier : {"Ali", "Sara", "Lina", "Karim", "Omar", "Yasmine"}  
Uniquement Python : {"Ali", "Karim"}
```

Exemple — Données & résultat attendu**3.2 — a_des_doublons(liste)**

```
liste_1 = ["Ali", "Sara", "Lina"]  
liste_2 = ["Ali", "Sara", "Ali"]
```

Résultat attendu :

```
a_des_doublons(liste_1) -> False  
a_des_doublons(liste_2) -> True
```

Exemple — Données & résultat attendu**3.3 — Set unique à partir de listes imbriquées**

```
tags_articles = [  
    ["python", "web", "api"],  
    ["python", "data"],  
    ["web", "css"],  
]
```

Résultat attendu :

```
{"python", "web", "api", "data", "css"}
```

Exemple — Données & résultat attendu**3.4 — Limite des sets : éléments non hashables**

```
coordonnees = {[1, 2], [3, 4]} # leve une TypeError
```

Comportement attendu : ce code lève `TypeError: unhashable type: 'list'`. Documentez en commentaire pourquoi, puis proposez l'alternative correcte :

```
coordonnees = {(1, 2), (3, 4)} # fonctionne : les tuples sont hashables
```

4 Bloc 4 — Combinaison listes / dicts / sets (40 min — 30 pts)

Bloc 4 : Combinaison des structures

- ☐ Agrégation d'une liste de ventes vers un dictionnaire de totaux
- ☐ Recherche du produit le plus vendu
- ☐ Extraction d'un set de produits distincts
- ☐ Fusion de deux inventaires avec addition des quantités communes
- ☐ Mini-challenge final combinant listes, dicts imbriqués et sets

Exemple — Données & résultat attendu**4.1 — Analyse des ventes**

```
ventes = [  
    {"produit": "pommes", "montant": 120},  
    {"produit": "bananes", "montant": 80},  
    {"produit": "pommes", "montant": 45},  
    {"produit": "oranges", "montant": 60},  
    {"produit": "bananes", "montant": 30},  
]
```

Résultats attendus :

```
Total par produit : {"pommes": 165, "bananes": 110, "oranges": 60}  
Meilleur produit : pommes (165)  
Produits distincts : {"pommes", "bananes", "oranges"}
```

Exemple — Données & résultat attendu**4.2 — fusionner_inventaires(inv1, inv2)**

```
inv1 = {"pommes": 20, "bananes": 15}  
inv2 = {"bananes": 10, "kiwis": 5}
```

Résultat attendu :

```
{"pommes": 20, "bananes": 25, "kiwis": 5}
```

Exemple — Données & résultat attendu**4.3 — Mini-challenge final**

```
etudiants = [  
    {"nom": "Ali", "matieres": {"maths": 14, "physique": 12}},  
    {"nom": "Sara", "matieres": {"maths": 18, "physique": 16, "svt": 15}},  
    {"nom": "Lina", "matieres": {"maths": 9, "physique": 11}},  
]
```

Résultats attendus :

```
Moyenne par etudiant :  
Ali : 13.0  
Sara : 16.33  
Lina : 10.0  
  
Matières enseignées (set) : {"maths", "physique", "svt"}  
  
Notes par matière :  
maths : [14, 18, 9]  
physique : [12, 16, 11]  
svt : [15]  
  
Meilleure matière (moyenne globale) : svt (15.0)
```

5 Règles de Gestion

Listes

- Une liste peut contenir des doublons ; l'ordre des éléments est conservé
- Le filtrage et le tri ne doivent jamais modifier la liste d'origine sans que ce soit explicitement demandé

Dictionnaires

- Une clé de dictionnaire est unique ; toute écriture sur une clé existante écrase l'ancienne valeur
- Toute opération de vente ou de mise à jour de stock doit vérifier la disponibilité avant modification
- Une fusion de deux dictionnaires ne doit jamais écraser silencieusement une donnée : la stratégie (somme, priorité, refus) doit être explicite dans le code

Sets

- Un set ne contient pas de doublons et n'est pas ordonné
- Seuls des éléments hashables (nombres, chaînes, tuples) peuvent être ajoutés à un set ; les listes et dictionnaires en sont exclus

6 Jeu de données de test (exemple global)

Jeu de données combiné, prêt à copier-coller, pour tester l'ensemble des blocs sur un même scénario.

Données d'entrée

```
stock_test = {"pommes": 50, "bananes": 0, "oranges": 12, "kiwis": 8}

commandes_test = [
    {"client": "Ali", "produit": "pommes", "quantite": 5},
    {"client": "Sara", "produit": "oranges", "quantite": 3},
    {"client": "Ali", "produit": "kiwis", "quantite": 2},
]

atelier_python_test = ["Ali", "Sara", "Lina", "Karim"]
atelier_java_test = ["Sara", "Omar", "Lina", "Yasmine"]

ventes_test = [
    {"produit": "pommes", "montant": 120},
    {"produit": "kiwis", "montant": 90},
    {"produit": "pommes", "montant": 60},
]
```

7 Structure attendue du script

```
challenge_structures.py
|
|-- notes_au-dessus(notes, seuil)
|-- a_des_doublons(liste)
|-- vendre(stock, produit, quantite)
|-- produits_epuises(stock)
|-- fusionner_inventaires(inv1, inv2)
|-- regrouper_par_mention(resultats) # reutilisable du bloc 2/4
|-- analyser_ventes(ventes) # bloc 4
```

```
-- mini_challenge_etudiants(etudiants) # bloc 4
```

8 Livrables attendus

1. Un script `.py` unique, fonctionnel de bout en bout pour les 4 blocs
2. Chaque fonction testée avec au moins le jeu de données fourni dans son encadré « Exemple »
3. Une exécution finale démontrant le mini-challenge du bloc 4 (4.3) de bout en bout